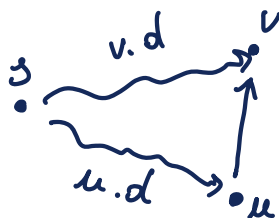


Lezione 13

Perché le procedure relax?

$$(u, v) \in E$$



$$\begin{array}{l} ? \\ v.d \geq u.d + w(u, v) \\ \downarrow \text{se si} \\ v.d = u.d + w(u, v) \end{array}$$

NOTAZIONE: $v.d$ = stima attuale del peso del cammino minimo da s a v

$\delta(s, v)$ = peso del cammino minimo da s a v .

13.1 PROPRIETÀ DEI CAMMINI MINIMI E DEI RILASSAMENTI

- DISUGUAGLIANZA TRIANGOLARE : $\forall (u, v) \in E, \delta(s, v) \leq \delta(s, u) + w(u, v)$
DEI CAMMINI MINIMI
- UPPER-BOUND PROPERTY: Per tutto il tempo della computazione abbiamo $v.d \geq \delta(s, v) \quad \forall v \in V$ e quando $v.d = \delta(s, v)$ non cambia più.
- NO-PATH PROPERTY: Se non ci sono cammini da s a v , $\delta(s, v) = \infty$.
- CONVERGENCE PROPERTY: Se $s \xrightarrow{p} u \rightarrow v$ è un cammino minimo in G per qualche $u, v \in V$ e $u.d = \delta(s, u)$ a qualunque istante precedente $\text{RELAX}(u, v, w) \Rightarrow v.d = \delta(s, v)$ a tutti gli istanti successivi.
- PREDECESSOR-SUBGRAPH PROPERTY: una volta che $v.d = \delta(s, v) \quad \forall v \in V$, il grafo dei predecessori è un albero dei cammini minimi radicato ad s .
- PATH-RELAXATION PROPERTY: se $p = \langle v_0, v_1, \dots, v_k \rangle$ da $s = v_0$ a v_k e se gli archi di p vengono rilassati secondo l'ordine $((v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k))$ allora $v_k.d = \delta(s, v_k)$ a prescindere dagli altri passi.

di valore che posso ottenere, anche se infimamente da valore degli archi di p .

13.2 CORRETTEZZA DELL'ALGORITMO DI BELLMAN-FORD

Lemura: Sia $G=(V,E)$ un grafo orientato, $s \in V$ un vertice sorgente, $w: E \rightarrow \mathbb{R}$ funzione peso.

Assumiamo che G non contenga cicli di peso negativo raggiungibili da s .

Allora, dopo $|V|-1$ iterazioni del ciclo for (righe 2-4) dell'algoritmo di Bellman-Ford

$$v.d = \delta(s, v), \quad \forall v \text{ raggiungibile da } s.$$

Proof: Utilizziamo la PATH-RELAXATION PROPERTY.

Si consideri v un qualunque vertice raggiungibile da s e sia $p = \langle u_0, u_1, \dots, u_k \rangle$ un cammino minimo con $u_0 = s$ e

$u_k = v$ da s a v .

Poiché i cammini minimi sono semplici, p ha al più $|V|-1$ archi, pertanto $k \leq |V|-1$.

Ognuna delle $|V|-1$ iterazioni del ciclo for (linee 2-4) rilassa tutti gli archi di E . Tra gli archi rilassati alle i -esime

iterazione c'è sicuramente (u_{i-1}, u_i) e questo vale per ogni $i \in \{1, \dots, k\}$. Per la PATH-RELAXATION PROPERTY allora

$$v.d = u_k.d = \delta(s, u_k) = \delta(s, v). \quad \square$$

COROLLARIO: Sia $G=(V,E)$ grafo orientato, $s \in V$ sorgente, $w: E \rightarrow \mathbb{R}$ funzione peso. Allora, $\forall v \in V$

$\exists$ un cammino da s a $v \iff$ terminare con $v.d < \infty$.

Proof: ESERCIZIO (super facile).

THM (CORRETTEZZA DI BELLMAN-FORD): Supponiamo di eseguire BELLMAN-FORD su (G, s, w) , dove $G=(V, E)$ grafo orientato, $s \in V$ sorgente, $w: E \rightarrow \mathbb{R}$ funzione peso.

Se non esistono cicli di peso negativo raggiungibili da s , allora:

1. l'algoritmo restituisce TRUE,
2. $v.d = \delta(s, v) \quad \forall v \in V$,
3. il sottografo dei predecessori G_π è un albero dei cammini naturali radicato ad s .

Se esiste un ciclo di peso negativo raggiungibile da s allora l'algoritmo restituisce FALSE.

PROOF: Se G non contiene cicli di peso negativo raggiungibili dalla sorgente s .

(2): Claim (*): al termine dell'algoritmo, $v.d = \delta(s, v) \quad \forall v \in V$

in fatti: • se v è raggiungibile da s , per il Lemma $v.d = \delta(s, v)$
 • se v non è raggiungibile da s , per il corollario $v.d = \infty$, inoltre $\delta(s, v) = \infty$ per la no-path property

quindi $v.d = \delta(s, v)$.

(3): La PREDECESSOR-SUBGRAPH property assieme al claim (*) implica che G_π è un albero di cammini naturali radicato ad s .

(1): Ora osserviamo che al termine dell'esecuzione $\forall (u, v) \in E$

$$v.d = \delta(s, v) \leq \delta(s, u) + w(u, v) \quad \left(\begin{array}{l} \text{disuguaglianza} \\ \text{triangolare} \\ \text{dei cammini} \end{array} \right)$$

quindi nessuno dei test alla linea 5 ha successo

$$\text{per } (u, v) \in E \quad v.d > u.d + w(u, v)$$

e quindi l'algoritmo restituisce TRUE.

Se G contiene un ciclo di peso negativo raggiungibile da s .

Sia esso $c = \langle v_0, v_1, \dots, v_k \rangle$ con $v_0 = v_k$

$$w(c) = \sum_{i=1}^k w(v_{i-1}, v_i) < 0.$$

Supponiamo per assurdo che l'algoritmo restituisca TRUE
 questo vuol dire che il test $v.d > u.d + w(u, v)$ non è mai
 soddisfatto,
 (linea)

In particolare, per i vertici del ciclo: $\forall i = \{1, \dots, k\}$
 $v_i.d \leq v_{i-1}.d + w(v_{i-1}, v_i)$

Sommiamo membro a membro per $i = 1, \dots, k$

$$\sum_{i=1}^k v_i.d \leq \sum_{i=1}^k [v_{i-1}.d + w(v_{i-1}, v_i)]$$

$$\sum_{i=1}^k v_i.d \leq \sum_{i=1}^k v_{i-1}.d + \sum_{i=1}^k w(v_{i-1}, v_i)$$

Poiché $v_k = v_0$, ogni vertice di c è contato esattamente
 una volta nelle due somme, ovvero

$$\underbrace{\sum_{i=1}^k v_i.d}_{v_1.d + v_2.d + \dots + v_k.d} = \underbrace{\sum_{i=1}^k v_{i-1}.d}_{v_0.d + v_1.d + \dots + v_{k-1}.d}$$

Inoltre, $\sum_{i=1}^k v_i.d \neq \infty$ perché $v_i.d \neq \infty$ e il ciclo c

è raggiungibile da s
 perciò lo sono tutti i suoi vertici.

COROLLARIO
 se v è raggiungibile
 da $s \Rightarrow v.d < \infty$

$$\sum_{i=1}^k v_i \cdot d \leq \sum_{i=1}^k v_{i-1} \cdot d + \sum_{i=1}^k w(v_{i-1}, v_i) \Rightarrow \sum_{i=1}^k w(v_{i-1}, v_i) \geq 0$$

assurdo perché $\sum_{i=1}^k w(v_{i-1}, v_i) < 0$.

L'assurdo deriva dall'aver supposto che l'algoritmo restituisce TRUE $\Rightarrow$ esso restituisce FALSE. $\square$

13.3 L'ALGORITMO DI DIJKSTRA

Risolve il Single-Source Shortest Path Problem nel caso in cui $w: E \rightarrow \mathbb{R}_0^+$, ovvero $w(u, v) \geq 0 \quad \forall (u, v) \in E$.

Con una buona implementazione, il tempo computazionale dell'algoritmo di Dijkstra è inferiore a quello dell'algoritmo di Bellman-Ford.

Possiamo pensare all'algoritmo di Dijkstra come a una generalizzazione della ricerca BFS ai grafi pesati.

Esso mantiene un insieme S di vertici i cui costi dei cammini minimi da s sono già stati determinati.

L'algoritmo seleziona ripetutamente il vertice $u \in V \setminus S$ con la stima del cammino minimo più piccola, aggiunge u ad S e rilassa tutti gli archi uscenti da u .

DISKSTRA (G, s, w)

1. INITIALIZE - SINGLE-SOURCE (G, s)

2. $S := \emptyset$

3. $Q := \emptyset$

4. for each vertex $v \in V$

5. INSERT (Q, v)

6. while $Q \neq \emptyset$

7. EXTRACT-MIN (Q) $\rightarrow u$ take the u of the smallest

8. $S = S \cup \{u\}$

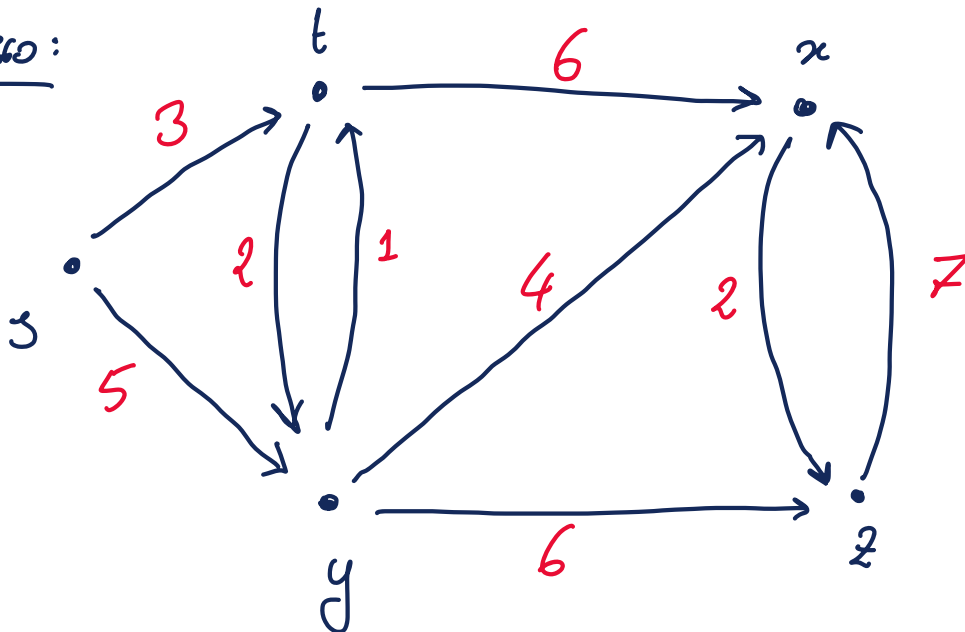
9. for each $v \in \text{Adj}[u]$

10. RELAX (u, v, w)

11. if the call of RELAX decreased $v.d$

12. DECREASE-KEY ($Q, v, v.d$)

Exercise:



INITIALIZATION

$s.d = 0$

$s.\pi = \text{NIL}$

$t.d = \infty$

$t.\pi = \text{NIL}$

$x.d = \infty$

$x.\pi = \text{NIL}$

$y.d = \infty$

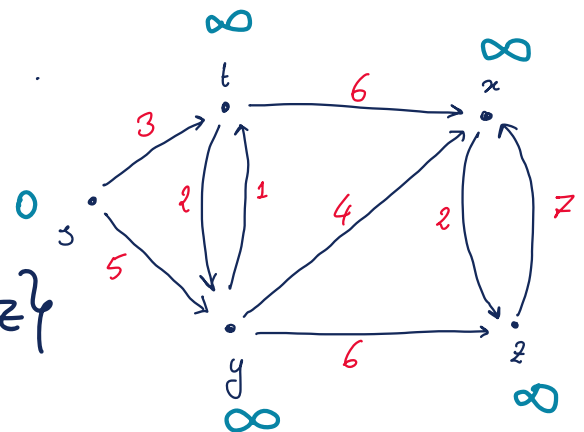
$y.\pi = \text{NIL}$

$z.d = \infty$

$z.\pi = \text{NIL}$

$S = \emptyset$

$Q = \{s, t, x, y, z\}$

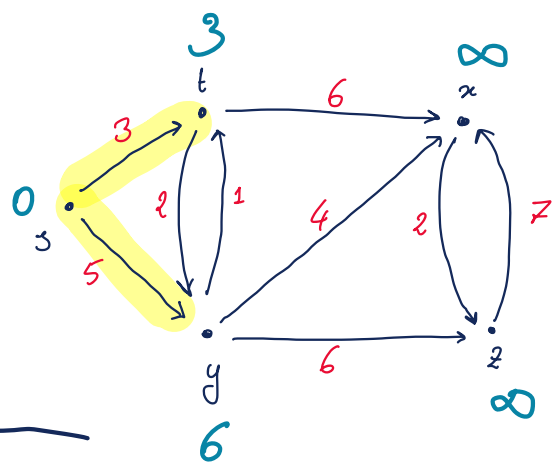


- $S = \{u\}$ $Q = \{t, x, y, z\}$

$$\text{Adj}[u] = \{t, y\}$$

$$t: \infty > 0 + 3 \Rightarrow t.d = 3, t.\pi = u$$

$$y: \infty > 0 + 6 \Rightarrow y.d = 6, y.\pi = u$$

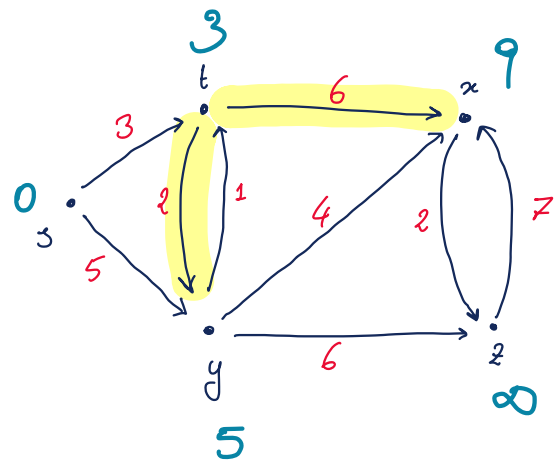


- $S = \{u, t\}$ $Q = \{x, y, z\}$

$$\text{Adj}[t] = \{x, y\}$$

$$x: \infty > 3 + 6 \Rightarrow x.d = 9, x.\pi = t$$

$$y: 6 > 3 + 2 \Rightarrow y.d = 5, y.\pi = t$$



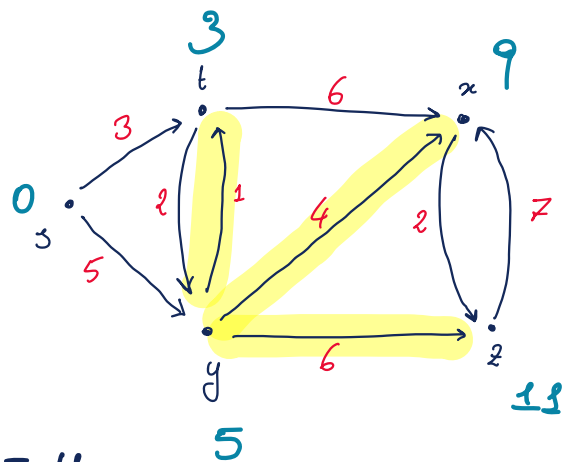
- $S = \{u, t, y\}$ $Q = \{x, z\}$

$$\text{Adj}[y] = \{t, x, z\}$$

$$t: 3 > 5 + 1 \text{ (NO)}$$

$$x: 9 > 5 + 4 \text{ (NO)}$$

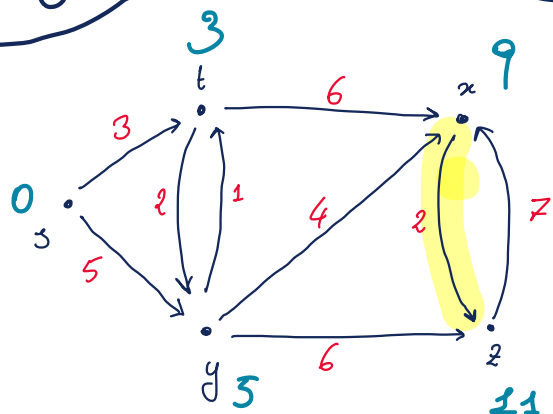
$$z: \infty > 5 + 6 \Rightarrow z.d = 11, z.\pi = y$$



- $S = \{u, t, y, x\}$ $Q = \{z\}$

$$\text{Adj}[x] = \{z\}$$

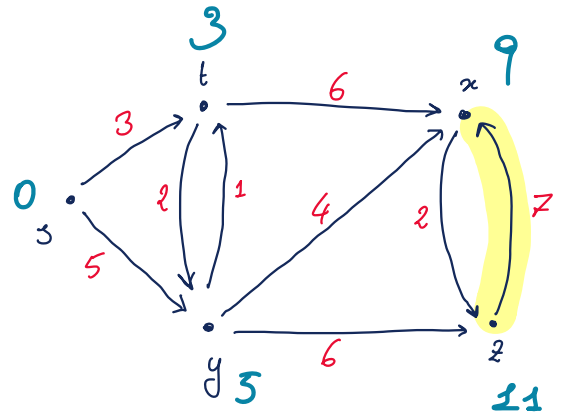
$$z: 11 > 9 + 2 \text{ (NO)}$$



• $S = \{u, t, y, x, z\}$ $Q = \{\}$

$\text{Adj}[z] = \{x\}$

$x: 9 > 11 + 7$ (no)



$\delta(s, s) = 0$

$\delta(s, t) = 3$

$\delta(s, y) = 5$

$\delta(s, x) = 9$

$\delta(s, z) = 11$

